

使用 Gemini 和 Cloud SQL pgvector，以視覺化方式呈現 AI 助理記憶體

來源：<https://codelabs.developers.google.com/next26/postgres-visual-memory-agent?hl=zh-tw>

步驟數：9

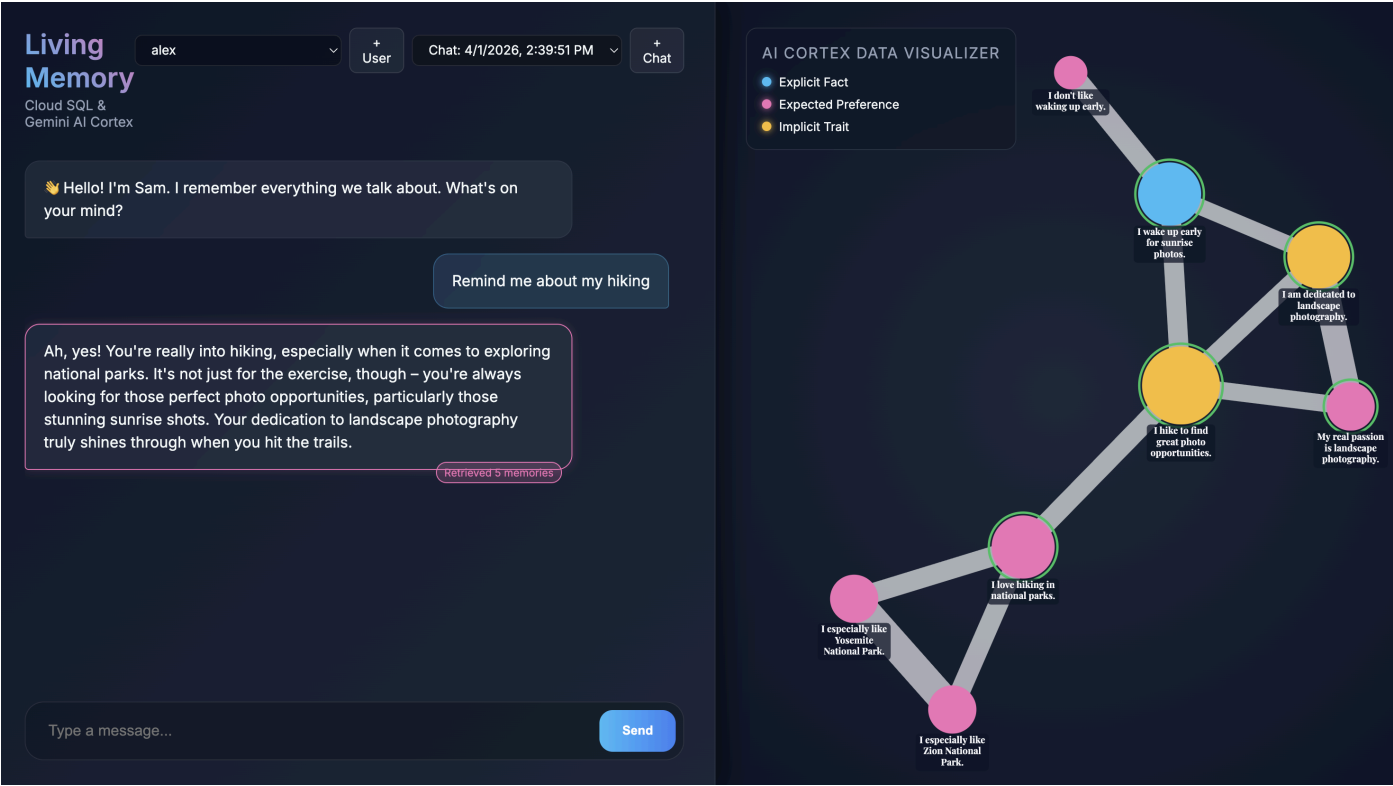
在本程式碼研究室中，您將瞭解如何使用 Gemini 和 Cloud SQL (含 pgvector) 建構「Living Memory」AI 助理。

目錄

1. [簡介](#)
2. [事前準備](#)
3. [複製示範存放區](#)
4. [建立及設定 Cloud SQL 資料庫](#)
5. [瞭解如何使用 pgvector 進行語意擷取](#)
6. [瞭解記憶體擷取功能](#)
7. [執行即時通訊應用程式](#)
8. [清除所用資源](#)
9. [恭喜](#)

1. 簡介

在本程式碼研究室中，您將瞭解如何建構 **Living Memory Demo**，這項 AI 輔助功能會追蹤對話中的「記憶」，提供個人化體驗。

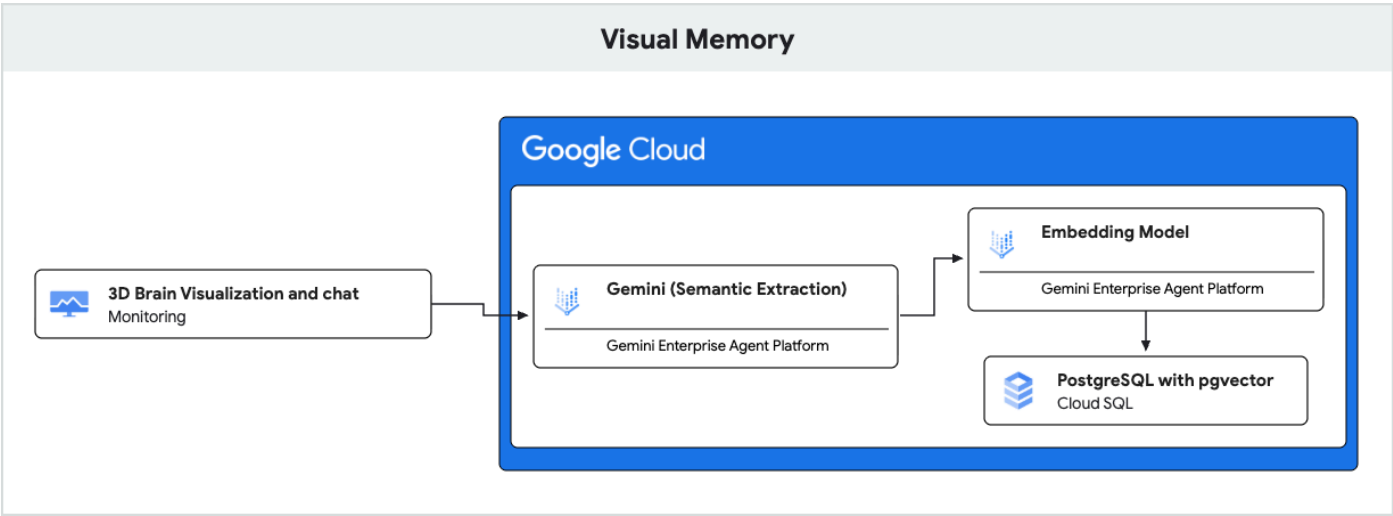


應用程式會使用 **Gemini** 進行自然語言理解，並使用 **PostgreSQL 適用的 Cloud SQL** 搭配 **pgvector 擴充功能**，根據語意相似度儲存及擷取這些記憶。

本程式碼研究室適合對 AI 和資料庫有興趣的各程度開發人員，大約 60 分鐘即可完成。建立的資源費用應低於 \$5 美元。

學習內容

- 如何設定支援 **pgvector** 的 PostgreSQL 適用的 Cloud SQL 執行個體。
- 如何使用 Gemini 從使用者訊息中以互動方式擷取「回憶」。
- 瞭解如何在 PostgreSQL 中執行向量搜尋，擷取 AI 回覆的相關內容。



軟硬體需求

- 已啟用計費功能的 Google Cloud 專案。
 - 具備指令列和 Node.js 的基本知識。
-

步驟 2 · 約 5 分鐘

2. 事前準備

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

專案設定

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

□□□

```
gcloud auth list
```

3. 確認專案已設定完成：

□□□

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

□□□

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用 API

在 [Cloud Shell](#) 執行下列指令，啟用必要的 API：

```
gcloud services enable sqladmin.googleapis.com \
    aiplatform.googleapis.com
```

步驟 3 · 約 5 分鐘

3. 複製示範存放區

現在，請取得 Living Memory 試用的程式碼。

1. 將存放區複製到 Cloud Shell 殼層環境：

```
git clone https://github.com/GoogleCloudPlatform/devrel-demos.git
cd devrel-demos/codelabs/visual-memory-postgres-demo
```

2. 安裝依附元件：

```
npm install
```

4. 建立及設定 Cloud SQL 資料庫

在本節中，您將建立 Cloud SQL 執行個體、初始化資料庫，並設定結構定義。

- 應用程式會使用環境變數進行設定。在 Cloud Shell 終端機中執行下列程式碼區塊，為這個工作階段設定必要變數：

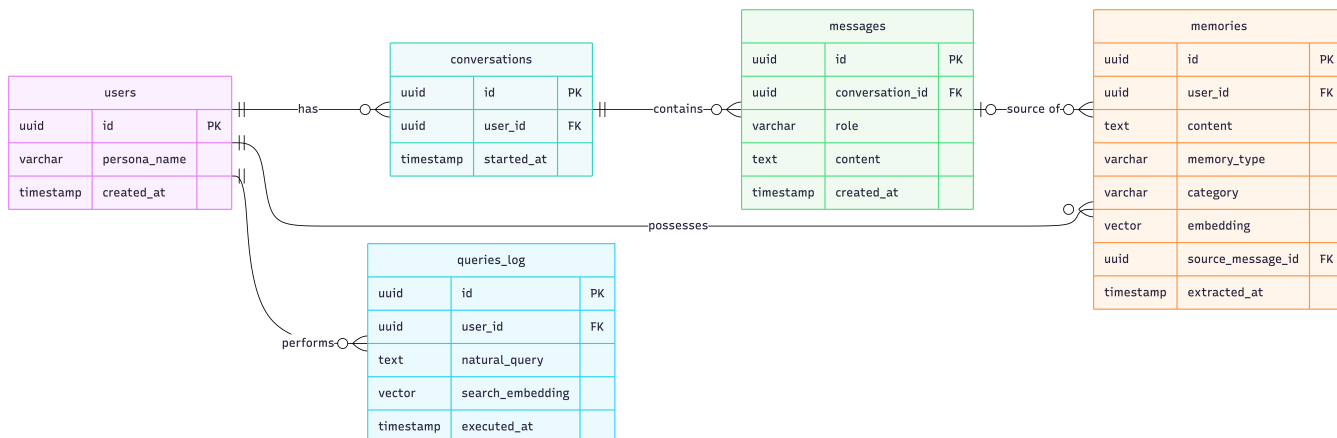
```
export REGION="us-central1"
export INSTANCE_NAME="living-memory-db"
export DB_HOST=127.0.0.1
export DB_PORT=5432
export DB_USER=memory_app
export DB_PASS=memory_app_password
export DB_NAME=living_memory
export PGPASSWORD=$DB_PASS
```

應用程式會使用 [Gemini Enterprise Agent Platform](#) 進行嵌入和對話，並使用 Cloud Shell 的應用程式預設憑證自動驗證。

- 建立執行個體。這個步驟通常需要 5 到 10 分鐘。

```
gcloud sql instances create $INSTANCE_NAME \
  --database-version=POSTGRES_16 \
  --cpu=1 \
  --memory=3840MB \
  --region=$REGION \
  --root-password=$DB_PASS \
  --edition=ENTERPRISE
```

系統正在建立執行個體，建議您在這段時間瞭解要使用的資料庫結構定義。指令碼會啟用 `vector` 擴充功能，並建立多個表格來支援應用程式：



- users**、**conversations**、**messages**：用於儲存使用者個人資料和對話記錄的標準資料表。
- memories**：這是檢索增強生成 (RAG) 的核心表格。每個資料列代表從對話中擷取的資訊 (例如「使用者喜歡健行」)。儲存的資料包括：
 - content**：記憶內容的文字。

- `memory_type`：記憶體類型 (`FACT`、`PREF` 或 `IMPLICIT`)。
- `embedding`：包含內容語意表示法的 768 維度 `vector` 資料欄，由 Gemini 生成。
- **pgvector 索引**：系統會在 `embedding` 資料欄上建立 [HNSW \(階層式可導覽小型世界\) 索引](#)。這對最佳化 k 最鄰近 (k-NN) 搜尋至關重要，可讓 `pgvector` 使用餘弦距離運算子 (`<=>`) 快速找出語意最相似的記憶內容。

3. 建立資料庫

```
gcloud sql databases create $DB_NAME --instance=$INSTANCE_NAME
```

4. 建立應用程式使用者

```
gcloud sql users create $DB_USER --instance=$INSTANCE_NAME --password=$DB_PASS
```

5. 啟動 Cloud SQL 驗證 Proxy。透過 Proxy 安全地存取執行個體，不需要設定 IP 許可清單。

```
(cloud-sql-proxy ${GOOGLE_CLOUD_PROJECT}:us-central1:living-memory-db &) && sleep 2 &&
echo ""
```

畫面上應會顯示類似「`The proxy has started successfully and is ready for new connections!`」的訊息。

如要進一步瞭解 Proxy，或查看在本機電腦 (Cloud Shell 以外) 上執行 Proxy 的操作說明，請參閱官方 [Cloud SQL Auth Proxy 說明文件](#)。

6. 套用 `schema.sql` 即可啟用 `vector` 擴充功能，並建立必要的資料表。由於 Proxy 正在執行，您現在可以透過 `127.0.0.1` 連線至執行個體。

```
psql -h 127.0.0.1 -U $DB_USER -d $DB_NAME < schema.sql
```

7. 確認結構定義建立成功。

```
psql -h 127.0.0.1 -U $DB_USER -d $DB_NAME -c "\dt"
```

您應該會看到輸出內容，列出 `conversations`、`memories`、`messages` 和 `users` 資料表。

```

              List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | conversations | table | memory_app
public | memories      | table | memory_app
public | messages      | table | memory_app
public | queries_log   | table | memory_app
public | users         | table | memory_app
(5 rows)
```


5. 瞭解如何使用 pgvector 進行語意擷取

在本節中，您將瞭解應用程式如何在生成回覆前，擷取 AI 的相關背景資訊。下列 `server.js` 的程式碼片段顯示 `/api/chat` 端點中負責這項作業的程式碼：

```
// Retrieve Similar Memories for Context (Using pgvector)
const promptEmbeddingRes = await ai.models.embedContent({
  model: 'gemini-embedding-001',
  contents: message,
  config: { outputDimensionality: 768 },
});

const promptEmbedding = promptEmbeddingRes.embeddings[0].values;
const embeddingStr = `[${promptEmbedding.join(',')}]`;

// Query DB for top 5 closest memories
const relevantMemories = await pool.query(
  `SELECT id, content, memory_type, category
   FROM memories
   WHERE user_id = $1
   ORDER BY embedding <=> $2::vector
   LIMIT 5`,
  [userId, embeddingStr]
);
```

運作方式

- **生成式 AI (嵌入)**：應用程式會接收使用者的收到的訊息，並使用 `gemini-embedding-001` 模型將文字轉換為 768 維度的向量。這個向量代表訊息的語意。
- **Cloud SQL (pgvector)**：應用程式會將該向量傳遞至 Cloud SQL。Cloud SQL 會使用 `pgvector` 擴充功能提供的 `<=>` (餘弦距離) 運算子，找出與提示語意最相似的 5 個記憶內容。
- **結果**：這是檢索增強生成 (RAG) 技術。AI 會從資料庫存取特定相關記憶內容，藉此提供個人化回覆，不必載入完整記錄。

步驟 6 · 約 5 分鐘

6. 瞭解記憶體擷取功能

接著，看看應用程式如何從對話中學習。以下程式碼片段來自 `server.js` 中的 `extractMemoriesAsync` 函式：

```

// MEMORY EXTRACTION LOGIC
async function extractMemoriesAsync(userMessage, userId, messageId) {
  const extractionPrompt = `
    Analyze the following user message. A memory profile is being built for this user.
    Extract ANY explicit facts (Facts), preferences (Pref), or implicit behavioral
    traits/styles (Implicit).
    Return the result as a raw JSON array of objects (NO Markdown blocks, just the JSON
    array).
    Format: [{"content": "string fact/sentence", "type": "FACT|PREF|IMPLICIT", "category":
    "General|Travel|Hobby|Persona"}]
    If nothing is found, return [].
    Message: "${userMessage}"
  `;

  const result = await ai.models.generateContent({
    model: 'gemini-2.5-flash',
    contents: extractionPrompt
  });
  let rawJson = result.text.replace(/```json/g, '').replace(/```$/g, '').trim();

  let extracted;
  try {
    extracted = JSON.parse(rawJson);
  } catch (e) {
    console.warn("Could not parse extracted JSON:", rawJson);
    return;
  }

  if (Array.isArray(extracted) && extracted.length > 0) {
    // Compute embeddings and save each to the DB
    for (const memory of extracted) {
      const embedRes = await ai.models.embedContent({
        model: 'gemini-embedding-001',
        contents: memory.content,
        config: { outputDimensionality: 768 },
      });
      const vectorData = `[${embedRes.embeddings[0].values.join(',')}]`;

      await pool.query(
        `INSERT INTO memories (user_id, content, memory_type, category, embedding,
source_message_id)
        VALUES ($1, $2, $3, $4, $5, $6)`,
        [userId, memory.content, memory.type.toUpperCase(), memory.category, vectorData,
messageId]
      );
      console.log(`Saved new memory: ${memory.content}`);
    }
  }
}

```

運作方式

- **生成式 AI (結構化輸出內容)**：應用程式會使用超快速的 `gemini-2.5-flash` 模型分析使用者訊息，並以 JSON 陣列的形式擷取結構化事實和偏好設定。
 - **Cloud SQL (混合式儲存空間)**：生成這些新事實的嵌入後，系統會將其儲存在 Cloud SQL 中。請注意，標準關聯式資料 (使用者 ID、文字內容、類別) 會與高維度向量資料一起儲存在單一資料列中。
 - **結果**：應用程式會運用 Gemini 的分析能力和 Cloud SQL 的儲存功能，即時建構可自動更新的記憶體設定檔。
-

步驟 7 · 約 5 分鐘

7. 執行即時通訊應用程式

1. 在資料庫中加入幾個範例使用者

```
npm run seed
```



CLOUD SHELL

Terminal

(n26-learn-data-cloud-screen-6) x + ▾



```
Database wiped perfectly. Starting seed process...
Loading persona pool from file...
Loaded 10 personas from file.

Creating User: Elena - The Busy Exec...
Sending intro message for Elena - The Busy Exec...
Saved new memory: Elena
Saved new memory: senior executive at a fast-paced tech company
Saved new memory: schedule is always packed
Saved new memory: starts days at 5 AM
Saved new memory: drinks black coffee
Saved new memory: runs 5 miles daily
Saved new memory: prefers dark mode on all devices
Saved new memory: uses dark mode to help with eye strain during long hours
Saved new memory: avid reader of non-fiction
Saved new memory: enjoys books on leadership and psychology
Saved new memory: travels frequently for business
Saved new memory: travels mostly to Europe and Asia
Saved new memory: allergic to peanuts
Saved new memory: has to be careful when eating out due to peanut allergy
Saved new memory: loves listening to classical music while working
Saved new memory: uses classical music to help focus
Saved new memory: drives an electric car
Saved new memory: passionate about sustainability
Saved new memory: prefers concise, bullet-point summaries in emails
Saved new memory: values efficiency above all else
Saved new memory: currently learning to play chess
Saved new memory: learning chess to improve strategic thinking
Intro message sent and memories extracted.
```

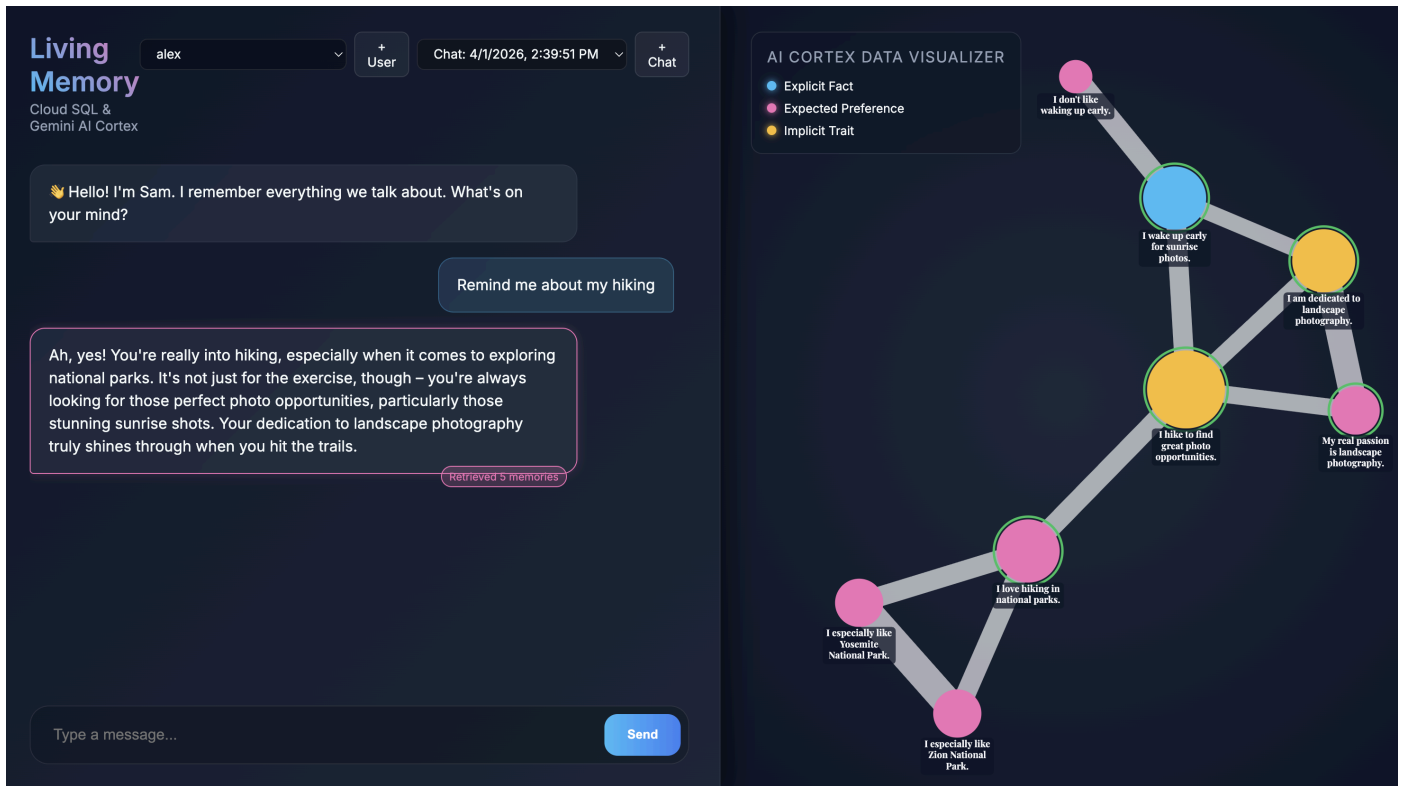
2. 然後執行伺服器

```
node server.js
```

3. 在 Cloud Shell 中，按一下終端機工具列右上方的「Web Preview」(網頁預覽)，然後選取「Change Port」(變更通訊埠)。輸入通訊埠編號 `3000`，然後按一下「變更並預覽」。

與助理互動

應用程式在瀏覽器中開啟後，您會看到 Living Memory 的即時通訊介面。右側的 **AI Cortex** 資料視覺化工具會以向量空間中的節點顯示記憶內容，並依類型 (事實、偏好、隱含特徵) 標示顏色。記憶體節點上的文字可能會很小，具體取決於螢幕解析度；如要放大檢視，請使用滑鼠或觸控板縮放及平移畫面。



查詢現有回憶

您先前執行的 `seed` 指令碼已建立兩個範例使用者，並預先填入一些回憶內容。

1. 在左上方的使用者下拉式選單中選取使用者。
2. 使用其中一個快速對話按鈕，或在對話輸入框中輸入 `Give me restaurant recommendations in New York City`，然後按下「傳送」。
3. 助理回覆後，你可以點選助理的訊息，查看助理使用了哪些記憶內容。這些字詞會以綠色醒目顯示，你可以縮放畫面，查看這些字詞如何協助生成回覆。

建立新使用者

現在，讓我們建立新使用者。

1. 按一下使用者下拉式選單旁的「+」按鈕，即可啟動新的即時通訊工作階段。
2. 使用系統產生的名稱和說明，或編輯這些內容來描述自己。
3. 按一下「建立」，應用程式就會開始擷取回憶。大約 30 秒後，右側的視覺化工具中應該會顯示新節點。這些是 Gemini 從訊息中擷取並儲存在 Cloud SQL 資料庫的事實和偏好設定。
4. 提出後續問題 (例如 `What food do I like?`)，看看 Google 助理是否會運用新近記憶內容進行對話。

步驟 8 · 約 3 分鐘

8. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請刪除您建立的資源。

1. 刪除 Cloud SQL 執行個體：

```
gcloud sql instances delete $INSTANCE_NAME --quiet
```

2. 移除示範存放區：

```
rm -rf ~/devrel-demos
```

步驟 9 · 約 1 分鐘

9. 恭喜

您已成功建構及部署「Living Memory」AI 助理！

目前所學內容

- 如何使用 **Cloud SQL pgvector** 進行語意搜尋。
- 如何使用 **Gemini** 動態擷取記憶體。

後續步驟

- 請參閱 [Cloud SQL pgvector 說明文件](#)。
- 進一步瞭解 [Gemini API 功能](#)。
- 深入瞭解 [Cloud SQL 驗證 Proxy](#)。
- 試著在 `server.js` 中自訂 `extractionPrompt`，擷取不同類型的資料！

盡情使用 Living Memory 打造回憶！
